School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory Assignment #11: Stored Routines, Triggers & JSON

Objectives:

- To write and call a stored procedure and a stored function.
- To keep stock in step automatically with triggers, and call routines from a Node.js web app.
- To store and query semi-structured data with a JSON column.

Scenario: your bookstore app is moving its stock rule into the database. Placing an order should go through a stored procedure, a trigger should reduce the book's stock (and refuse an order when there is not enough), and cancelling an order should give the stock back automatically. You will also show each customer's order count with a stored function, and add a flexible JSON column for each book's format and number of pages. Everything you build appears in the bookstore web app, exactly as it did for the university app in the lab exercises.

Setup

Download and unzip **Lab11_Assignment_Starter.zip**. In MySQL Workbench, run `sql/bookstore_setup.sql` once. It creates a separate `bookstore_lab11` database, so your earlier bookstore data is left untouched:

```
books(     id, title, author, price, stock )
customers( id, name, email )
orders(    id, customer_id, book_id, quantity )   -- FKs to customers and books

-- today: routines and triggers run in the database; JSON adds flexible fields
```

Open the **`.env`** file and set `DB_USER` and `DB_PASSWORD` to your own MySQL login (`DB_NAME` is already `bookstore_lab11`). As in Lab 10, `db.js` reads them with `dotenv`. Then, in a terminal in the project folder:

```
npm install      # once: downloads express, mysql2 and dotenv
npm run dev      # starts the server on http://localhost:3000 (Ctrl + C stops it)
```

Open **http://localhost:3000**. The web app shows the **Books** with their stock (and, after Q7, their format and pages with a format filter), the **Customers** with their order count, a **Place / cancel an order** panel and a **Responses** log. The log also shows how the book's stock changed after each order or cancel, and flags it in red when the change is not exactly the quantity. A yellow note in each panel tells you which question fills it in.

Q1–Q4 are SQL: write them in `sql/q1_q4_routines.sql` and run them in Workbench (keep the `DELIMITER //` lines around routines and triggers). The order routes in `controllers/orderController.js` are the Lab 9 transactions. They work now, but once your Q3

and Q4 triggers exist they change the stock **twice**; Q5 fixes that. Ready-made test requests are in `requests.http`. This assignment is marked out of **12**.

Questions

1. **Stored procedure.** Write a procedure `place_order(IN p_customer INT, IN p_book INT, IN p_qty INT)` that inserts one row into orders. CALL it to create an order for customer 4 (book 2, quantity 1) and show the new row. **(1.5 points)**
2. **Stored function.** Write a function `order_count(p_customer INT) RETURNS INT` that returns how many orders a customer has (remember `READS SQL DATA`). Use it in a SELECT that lists every customer with their order count. **(1.5 points)**
3. **Trigger with a stock guard.** Create an AFTER INSERT trigger `after_order_insert` on orders that reduces the book's stock by `NEW.quantity`, but only `WHERE stock >= NEW.quantity`. If no row was updated (`ROW_COUNT() = 0`), stop the order with `SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'not enough stock'`. Show that an order reduces the stock with no extra query, and that ordering more than the stock fails (Error 1644) and creates no order. **(1.5 points)**
4. **Trigger (reverse).** Add an AFTER DELETE trigger `after_order_delete` on orders that adds `OLD.quantity` back to the book's stock. Demonstrate that deleting an order restores the stock. **(1.5 points)**
5. **Use the routines from the web app.** In `controllers/orderController.js`, rewrite `POST /customers/:id/orders` so that, after the existing 400 check, it makes **one** call: `pool.execute('CALL place_order(?, ?, ?)', [...])`, answers **201**, maps your SIGNAL (`err.sqlState === '45000'`) to **409** and a missing customer or book to **404**. In `DELETE /customers/:id/orders/:orderId` keep only the DELETE (404 when nothing was deleted). In `controllers/customerController.js` add `order_count(id) AS orders`. In the web page, show one order and one cancel that each change the stock by exactly the quantity, and one order that is refused with 409. **(1.5 points)**
6. **JSON column.** In `sql/q6_q7_json.sql`, add a JSON column `attributes` to books and store an object `{format, pages}` for **every** book with `JSON_OBJECT` (format is hardcover, paperback or ebook; make at least two ebooks). Show the rows. **(1.5 points)**
7. **Query JSON.** In Workbench, write a query that returns each book's title and its format, and a second query that returns only the books whose format is 'ebook' (use `->>` or `JSON_EXTRACT`). Then make `GET /books` in `controllers/bookController.js` return `format` and `pages`, and support `GET /books?format=ebook` with a `?` parameter. Show the Format filter working in the web page. **(1.5 points)**
8. **Short answer.** In two or three sentences each: (a) give one good use for a trigger and one risk of using triggers, referring to what you saw before you fixed Q5; and (b) explain when you would choose a JSON column over adding ordinary columns or a new table. **(1.5 points)**

Submission

- Zip your project folder **without** the `node_modules` folder and **without** your `.env` (keep `.env.example`). It must include your completed `sql/q1_q4_routines.sql` and `sql/q6_q7_json.sql`.

- One PDF with your screenshots (Q1–Q7) and your answers to Q8. Each screenshot should show the SQL or request **and** the result: the result grid, or the web page with the Responses log.
- Due date: _____